

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES

Tema 0. Transición de Java a C#

HOLA MUNDO

- Ambos son lenguajes orientados a objetos.
- Tienen la misma sintaxis básica: declaración de variables, condicionales y los bucles son idénticos.
- El flujo del programa comienza en una clase con una función pública y estática llamada Main.

Java

```
• public class Hola {  
    - public static void main(String[] args) {  
        System.out.println("Hola mundo");  
    - }  
• }
```

C#

```
• using System;  
• class Hola {  
    - static void Main(string[] args) {  
        Console.WriteLine("Hola mundo");  
    }  
• }
```

VARIABLES Y TIPOS

- Muy similares.
- En C# existen más alias para tipos primitivos.

Java

- `int x = 10;`
- `double d = 3.14;`
- `boolean b = true;`
- `String s = "texto";`

C#

- `int x = 10;`
- `double d = 3.14;`
- `float f = 3.14f;`
- `bool b = true; // no "boolean"`
- `string s = "texto"; // "string" en minúscula`

CONDICIONALES Y BUCLES

- Prácticamente iguales.
- `for`, `while`, `do-while` funcionan igual.
- C# además tiene `foreach` para colecciones.

Java

```
• if (x > 0) {  
•     System.out.println("positivo");  
• }
```

C#

```
• if (x > 0) {  
•     Console.WriteLine("positivo");  
• }
```

FUNCIONES / MÉTODOS

- Convención: en C# los métodos suelen comenzar en mayúscula (PascalCase)

Java

```
• public static int suma(int a, int b) {  
•     return a + b;  
• }
```

C#

```
• public static int Suma(int a, int b) {  
•     return a + b;  
• }
```

CLASES Y OBJETOS

- Muy similares.

Java

```
• public class Persona {  
•     private String nombre;  
•     public Persona(String nombre) {  
•         this.nombre = nombre;  
•     }  
•     public String getNombre() {  
•         return nombre;  
•     }  
• }
```

C#

```
• public class Persona {  
•     private string nombre;  
•     public Persona(string nombre) {  
•         this.nombre = nombre;  
•     }  
•     public string GetNombre() {  
•         return nombre;  
•     }  
• }
```

PROPIEDADES

- En lugar de `get/set` explícitos, C# usa properties.

Java

- `getNombre() / setNombre()`

C#

- `public class Persona {`
- `public string Nombre { get; set; }`
- `}`

ARRAYS Y COLECCIONES

- En C#: `List<T>` está en `System.Collections.Generic`.

Java

- `int[] nums = new int[5];`
- `ArrayList lista = new ArrayList<>();`

C#

- `int[] nums = new int[5];`
- `List lista = new List();`

MANEJO DE CADENAS

- En C# muchas propiedades usan propiedades (``.Length``) en vez de métodos (``.length()``).

Java

- `String s = "Hola";`
- `System.out.println(s.length());`

C#

- `string s = "Hola";`
- `Console.WriteLine(s.Length);`

CHEAT SHEET COMPLETO DE MICROSOFT

C# for Java developers cheat sheet

C# for Java Developers

Microsoft
Visual Studio 2010

Keywords

Java	C#	Java	C#
abstract	abstract	native	extern
assert	Debug.Assert (method)	new	new
break	break	null	null
case	case	namespace	namespace
catch	catch	private	private
class	class	protected	internal
const	const	public	public
continue	continue	return	return
default	default	static	static
do	do	strictfp	n/a
else	else	super	base
enum	enum	switch	switch
extends	:	synchronized	lock
false	false	this	this
final	sealed	throw	throw
finally	finally	throw	throw
for	for / foreach	transient	(Nonserialized) (attribute)
goto	goto	true	true
if	if	try	try
implements	:	try	try
import	using	void	void
instanceof	is	volatile	volatile
interface	interface	while	while

Note: The **const** and **goto** keywords in Java have no function. The C# **const** and **goto** keywords are operational.

C# keywords not found in Java, or with different behavior

Keyword	Description	Keyword	Description
base	Provides access to members in a parent class	operator	Declares an overloaded operator
checked	Enables arithmetic overflow checking	out	Declares an output parameter (method call)
delegate	Declares a type-safe reference to a method	override	Declares a covariant type parameter (generic interface)
event	Declares an event	protected	Declares a method that overrides a method in a base class
explicit	Declares a narrowing conversion operator	readonly	Declares a member that is accessible only within the class and descendant classes (different semantics from Java protected keyword)
implicit	Declares a widening conversion operator	ref	Declares a reference to a value type
in	Declares a contravariant type parameter for a generic interface	struct	Defines a new value type
new	Declares a method that hides a method in a base class (method modifier)	virtual	Declares a member that can be overridden

Operators

Java	C#	Description
x.y	x.y	Member access 'dot' operator
f(x)	f(x)	Method invocation operator
a[x]	a[x]	Array element access operator
++, --	++, --	Increment and decrement operators (pre and post-fix)
new	new	Object instantiation operator
instanceof	is	Type verification operator
(T)x	(T)x	Explicit cast operator
x + y	x + y	Addition and subtraction operators (binary)
x - y	x - y	Positive and negative operators (unary)
! x	!	Logical negation operator
x & y	x & y	Conditional AND and OR operators (short-circuited evaluation)
x y	x y	Conditional AND, OR, and XOR operators (full evaluation of operands)
~ x	~	Bitwise complement operator
x & y	x & y	Bitwise AND, OR, and XOR operators
<< x, >> x	<< x, >> x	Signed left-shift and right-shift operators
>>> x	>>> x	Unsigned right-shift operator
x / y, x % y	x / y, x % y	Multiply, divide, and modulus operators
x == y, x != y	x == y, x != y	Is-equal-to and is-not-equal-to operators
x < y, x > y, x <= y, x >= y	x < y, x > y, x <= y, x >= y	Relational less-than, greater-than, less-than-or-equal-to, and greater-than-or-equal-to operators
x is y	x is y	Conditional operator
=	=	Assignment operator

C# operators not available in Java:

Operator	Description	Operator	Description
<<	Unsigned left-shift operator	default(T)	Returns the default value for a type (generic)
<<=	Unsigned left-shift compound assignment operator	sizeof(T)	Returns the size of a type
??	Null-coalescing operator	stackalloc	Allocates a block of memory on the stack
~>	Lambda expression operator	typeof(x)	Returns the type of an expression
as	Safe casting operator	unchecked	Disables arithmetic overflow checking
checked	Enables arithmetic overflow checking	unsafe	Enables unsafe code

C# identifiers that should not be used as names for types, methods, or variables:

dynamic	join	get
from	let	value
get	orderby	var
group	partial	where
into	select	yield

Common Datatypes

Java	C#	Java Range	C# Range
boolean	bool	true or false	true/false
byte	sbyte	-128 to 127	-128 to 127
char	char	0 to 2 ¹⁶ -1	0 to 2 ¹⁶ -1
double	double	±5.0 × 10 ⁻³² to ±1.7 × 10 ³⁰⁸	±5.0 × 10 ⁻³² to ±1.7 × 10 ³⁰⁸
float	float	±1.5 × 10 ⁻⁴⁵ to ±3.4 × 10 ³⁸	±1.5 × 10 ⁻⁴⁵ to ±3.4 × 10 ³⁸
int	int	2 ³¹ to 2 ³¹ -1	-2 ³¹ to 2 ³¹ -1
long	long	-2 ⁶³ to 2 ⁶³ -1	-2 ⁶³ to 2 ⁶³ -1
short	short	-2 ¹⁵ to 2 ¹⁵ -1	-2 ¹⁵ to 2 ¹⁵ -1
string	string	n/a	n/a
object	object	n/a	n/a
Date	DateTime	1st Jan 1970 to 31st Dec 9999	1st Jan 0001 to 31st Dec 9999

Datatype examples

Java	C#
int i = 1;	int i = 1;
byte b = 1;	byte b = 1;
double d = 1.1;	double d = 1.1;
long l = 1L;	long l = 1L;
short s = 1;	short s = 1;
boolean found = true;	bool found = true;
char c = 'x';	char c = 'x';
String title = "Hello World";	string title = "Hello World";

Array examples

Java	C#
int[] data1;	int[] data1;
int data2[];	n/a

C# datatypes not provided as part of Java

Type	Range	Size(bits)
decimal	28-29 significant figures	128
byte	0 to 255	8
uint	2 ³² -1	32
ulong	0 to 2 ⁶⁴ -1	64
ushort	0 to 2 ¹⁶ -1	16

Java wrapper classes mapped to C# structs

Java Wrapper	C# Struct
Boolean	Boolean
Byte	Byte
Character	Char
n/a	Decimal
Double	Double
Float	Float
Integer	Int32
Long	Int64
Short	Int16
Void	n/a

Development Areas



Windows

Windows Phone

Windows Azure



C# Features Not Available in Java 6

- LINQ
- Delegates
- Events
- Properties
- Indexers
- Lambda Expressions
- Partial Classes
- Dynamic Runtime Language
- Structures
- Operator Overloading
- Partial Methods
- Optional Parameters